

What a Blockchain Is

The problem digital money has to solve, and how a tamper-evident, replicated ledger with open membership solves it — the ideas underneath every full node.

SUBJECT hathor-core · Part I · Track B (domain concepts)

CHAPTER 06 · Foundations · Concepts

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Double-spend · Distributed ledger · Cryptographic hash · Hash pointer · Tamper-evidence · Sybil attack · Proof-of-work · Consensus · Probabilistic finality

Table of Contents

Chapter 6 — What a Blockchain Is	3
6.1 The problem: spending the same coin twice	3
6.2 The easy answer, and why it's rejected	4
6.3 The distributed ledger: everyone holds a copy	5
6.4 The primitive: cryptographic hash functions	5
6.5 From records to a tamper-evident chain	6
6.6 Who appends? Sybil attacks and proof-of-work	7
6.7 Finality is probabilistic	8
6.8 Bridge — how Hathor answers these questions	9
Recap	10

Chapter 6 — What a Blockchain Is

What you'll learn in this chapter

- The one hard problem digital money must solve — the **double-spend problem** — and why it is hard only when there is no central authority.
- Why the obvious fix (a trusted company's database) is rejected, and what is gained and lost by removing it.
- The **distributed ledger** idea: everyone keeps a copy, so no one owns the truth.
- The cryptographic primitive the whole edifice rests on — the **hash function** — built up from its properties, not its math.
- How hashes turn a list of records into a **tamper-evident chain**, so altering old history is detectable.
- The membership problem (**Sybil attacks**) and how **proof-of-work** answers "who gets to add the next block."
- Why finality is **probabilistic**, not absolute — and a **bridge** to how Hathor's specific design (a DAG, not a chain) re-answers each of these questions.

Track A taught you how the code is built. Track B turns to what it is about. This chapter is the foundation of the foundation: every later domain chapter — UTXO (Ch 7), the DAG (Ch 8), proof-of-work and weight (Ch 9), voiding and finality (Ch 10) — is a refinement of an idea introduced here. We build it from a single question and follow the consequences.

The body is framework-agnostic: we reason about any system that wants to be digital money without a central authority. Where Hathor's answer differs from the classic blockchain answer — and in several places it differs sharply — the closing Bridge (§6.8) says so and points to the chapter that covers it. Read this chapter for the problem; read the rest of Track B for Hathor's solution.

6.1 The problem: spending the same coin twice

Start with physical cash. When you hand someone a £10 note, two things happen at once: they now have it, and you no longer do. The physical object cannot be in two places. You cannot spend the same note twice, because giving it away is the same act as losing it.

Now try to make digital money. A digital coin is data — a file, a number, a record. And data has a property cash does not: **it copies perfectly and for free**. If a coin is just a file, then "sending" it to Alice means sending her a copy, and you still have the original to send to Bob. You could spend the same coin a thousand times. This is the **double-spend problem**¹, and it is the central problem of digital money. Everything else — the hashing, the chains, the consensus, the mining — exists to solve it.

State it precisely, because the precision matters: the problem is not "can I copy the coin" (of course you can copy data); it is "**can everyone agree on which transfer was the real one**." If the whole world agrees that the coin went to Alice and that the transfer to Bob is invalid, then your copy is worthless — agreement is what makes it money. So the double-spend problem is really a problem of **agreement**: getting many independent parties to share one consistent history of who owns what.

6.2 The easy answer, and why it's rejected

There is a trivial solution, and you use it every day: **put one trusted party in charge of the ledger**. Your bank keeps a database of balances. When you pay Alice, the bank subtracts from your row and adds to hers, in one atomic step, and refuses to do it twice. Double-spending is impossible because there is exactly one authoritative copy of the truth and one party allowed to write to it. This is how essentially all conventional digital payment works.

So why would anyone want anything else? Because that single trusted party is also a single point of control and failure:

- It can **ensor** — refuse your transaction, freeze your account.
- It can **err or be corrupted** — a bug, an insider, or a hack rewrites balances.
- It is a **single target** — one breach, one outage, one coerced operator compromises everyone.
- It demands **trust** — you must believe the operator is honest and competent, forever.

The motivating goal of a cryptocurrency is to get the double-spend guarantee without anointing such a party — to build a ledger that **no single entity owns, controls, or can quietly rewrite**. That is a much harder problem, and the rest of this chapter is the shape of its solution. Note the trade-off up front, because it is the whole bargain: you remove the trusted party and gain censorship-resistance and resilience; you pay for it with enormous complexity and, as we'll see, with speed and with absolute certainty. Whether that trade is worth it is a judgment call, not a technical fact — but understanding the machinery requires taking the goal as given.

6.3 The distributed ledger: everyone holds a copy

If no one party can hold the ledger, the answer is that **everyone holds it**. A **distributed ledger**² is a record of all transactions that is replicated across many independent computers (the nodes³ — the thing `hathor-core` is), each keeping its own full copy, with no master.

This immediately reframes the double-spend problem. There is no longer one database to write to; there are thousands of copies that must somehow **stay in agreement** about the same history. If you could convince half the network the coin went to Alice and the other half it went to Bob, you'd have spent it twice. So the engineering problem becomes:

1. How does a new transaction get **broadcast** to everyone? (The peer-to-peer network — Ch 34.)
2. How can each node **independently verify** a transaction is legitimate, trusting no one? (Verification — Ch 31.)
3. How do all the copies **agree on one ordering** of history, including which of two conflicting transactions is the real one? (Consensus — Ch 32, and §6.6–6.7 here.)
4. How is the agreed history made **tamper-evident**, so no node can secretly rewrite its copy and pass it off as true? (Hashing — §6.4–6.5 here.)

We take these in the order a newcomer needs them: first the tamper-evidence (it's the cryptographic core), then the agreement (it's the subtle part). Both rest on one primitive, so we define that first.

6.4 The primitive: cryptographic hash functions

Almost everything in a blockchain is built from one tool, the **cryptographic hash function**⁴. You do not need its mathematics to understand the system — you need its properties. So we define it by what it does.

A hash function takes an input of any size — a word, a file, an entire transaction — and produces a fixed-size output, called the **hash** or **digest**⁵ (in the systems we care about, 256 bits, written as 64 hexadecimal characters). Think of it as a machine that reads any amount of data and stamps out a short, fixed-length fingerprint of it.

"hello"	→ [hash fn] →	2cf24dba5fb0a30e26e83b2ac5b9e29e...
"hello."	→ [hash fn] →	e6f6a3d6... (utterly different)
<a 4 GB video>	→ [hash fn] →	9b2c8f01... (still just 64 hex chars)

Four properties make it useful, and each one earns its keep later:

1. **Deterministic.** The same input always yields the same hash. Hash "hello" on any machine, any day, and you get the identical 64 characters. This is what lets two nodes confirm they hold the same data: compare fingerprints, not the whole file.
2. **One-way (preimage resistance).** Given a hash, you cannot work backwards to recover the input. The fingerprint reveals nothing practical about what produced it. The only way to find an input with a given hash is to guess and check — which is exactly what mining will exploit (§6.6).
3. **Avalanche effect.** Change the input by one bit and the output changes completely and unpredictably — "hello" and "hello." share nothing in their hashes. There is no partial similarity, no gradient. This is what makes tampering detectable: you cannot make a small, hash-preserving edit.
4. **Collision resistance.** It is infeasible to find two different inputs that produce the same hash. So a hash is, in practice, a unique identifier for its input — if two things have the same hash, they are the same thing.

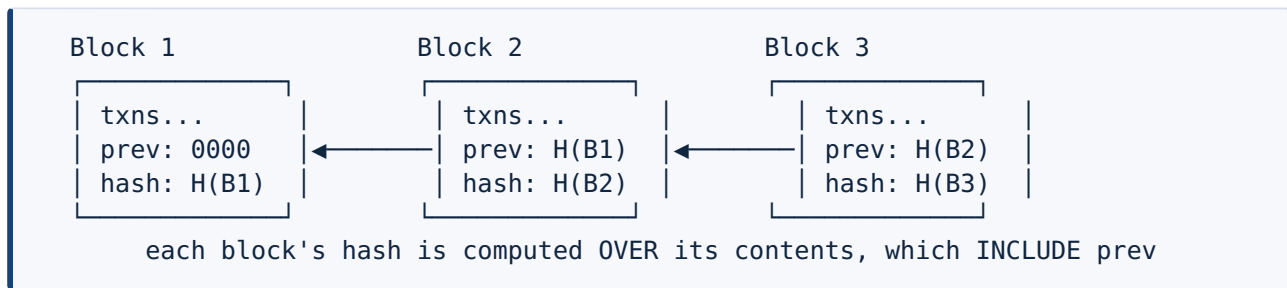
Put properties 3 and 4 together and you get the key consequence: **a hash is a tamper-evident fingerprint.** Publish the hash of a document, and anyone can later re-hash the document and check it matches; if even one character was altered, the hashes diverge and the tampering is exposed. We are about to use that to protect an entire history.

A hash is not encryption. A common confusion: encryption is reversible (with the key, you recover the original); hashing is one-way by design — there is no "unhash." A hash proves integrity ("this data is unchanged"); it does not hide data for later retrieval. Different tools, different jobs.

6.5 From records to a tamper-evident chain

Here is where "block" and "chain" finally enter, and where the classic design gets its name. We want an append-only history of transactions that anyone can verify and no one can secretly rewrite.

First, bundle transactions into **blocks**⁶ — batches of records grouped together, mostly so the network agrees on them a batch at a time rather than one transaction at a time. Now the trick: each block includes, as part of its own data, **the hash of the block before it.** That backward reference is a **hash pointer**⁷ — not just "the previous block is over there," but "the previous block had exactly this fingerprint."



Trace what this buys you. Suppose an attacker wants to alter one transaction in Block 1 — say, to erase a payment. The moment they change Block 1's contents, Block 1's hash changes (avalanche effect, §6.4). But Block 2 contains the old hash of Block 1 as its `prev` pointer — so Block 2 no longer points to the (now altered) Block 1; the link is visibly broken. To repair it, the attacker must update Block 2's `prev`, which changes Block 2's hash, which breaks Block 3's pointer, and so on. **One change anywhere forces re-writing every block after it.** The hash pointers weld the blocks into a structure where the most recent hash certifies the entire history behind it. This is a **hash chain**, and it is the literal "blockchain."

The chain makes tampering detectable and expensive-to-hide, but on its own it does not make tampering hard — re-computing a few hashes is fast. Two more pieces close the gap. First, the history is replicated (§6.3): to convince the network, you must alter not your copy but everyone's. Second — the heart of it — adding or rewriting a block is made deliberately, physically costly, so that out-running the honest network is infeasible. That cost is the consensus mechanism, and it answers the question we have been deferring: who gets to add the next block?

6.6 Who appends? Sybil attacks and proof-of-work

In a network with no central authority and open membership, anyone can join. That openness creates a specific attack. If "the majority decides which history is true" and votes are counted per participant, an attacker just creates a million fake participants and out-votes everyone. Cheaply manufacturing many identities to gain disproportionate influence is a **Sybil attack**⁸, and any open, permissionless system must defend against it. Counting noses doesn't work when noses are free.

The insight that cracked this — the founding idea of Bitcoin — is to make influence depend not on how many identities you have but on how much of some scarce, expensive resource you can prove you spent. If adding a block requires burning something costly, then controlling the history requires controlling a majority of that cost, and fake identities don't help — each fake still has to pay.

Proof-of-work⁹ is the classic choice of scarce resource: computation. To add a block, a participant (a **miner**¹⁰) must find a number — a **nonce**¹¹ — such that hashing the block together with that nonce produces a hash below some agreed **target**¹² (e.g. "the

hash must start with 20 zeros"). Because the hash is one-way and avalanche (§6.4), there is no clever way to find such a nonce — you can only **guess and check**, billions of times per second, until one works. That repeated guessing burns real electricity and time. The result is asymmetric in exactly the useful way:

- **Finding** a valid nonce is hard — it takes the whole network many guesses (tuned, by adjusting the target, to one block every so often).
- **Checking** a claimed nonce is trivial — hash once, compare to the target. Every node verifies a miner's work instantly.

So a miner who finds a valid block has demonstrably spent a large amount of computation, and anyone can confirm it for free. The hash below the target is the proof of the work. Now rewriting history has a price: to alter Block 1 and re-thread the chain (§6.5), an attacker must redo the proof-of-work for Block 1 and every block after it, faster than the honest network extends the chain ahead — which requires out-computing the rest of the world combined. The append-rule becomes "**the history with the most accumulated work wins**," and work cannot be faked, only paid for.

Why "the most work," not "the longest"? Loosely, people say "longest chain wins," but the precise rule is most cumulative work, because the difficulty target changes over time — a shorter chain of harder blocks can represent more total computation than a longer chain of easy ones. Hold this distinction; Hathor takes it much further by measuring work directly as weight (Ch 9).

Proof-of-work is not the only answer to the Sybil problem — **proof-of-stake** makes the scarce resource money at risk rather than computation, and **proof-of-authority** drops openness entirely in favor of a known set of signers (Hathor supports this last one for private networks — Ch 32). But proof-of-work is the original, and it is the one whose vocabulary — nonce, target, mining, difficulty — runs through every full node.

6.7 Finality is probabilistic

One consequence of "most-work-wins" surprises everyone at first: in this design, a transaction is **never absolutely final**. It is only ever increasingly unlikely to be reversed.

Here is why. Suppose two miners find a block at nearly the same moment, each extending the chain differently. The network temporarily splits — some nodes build on one, some on the other. This is a **fork**¹³. The tie breaks when the next block lands on one side, making that branch carry more work; the heavier branch wins and the other is abandoned. Any transaction that was only in the abandoned branch is undone — rolled back — an event called a **reorganization**, or **reorg**¹⁴.

So a transaction one block deep could still be reversed by a short reorg. But each additional block built on top of it is more work an attacker would have to out-pace to reverse it — so the probability of reversal shrinks exponentially with **depth**, the number of blocks piled on after it. This is **probabilistic finality**¹⁵ : you never get a mathematical guarantee that a payment is permanent, only ever-stronger confidence. (This is why exchanges "wait for N confirmations" — they are waiting for enough depth that a reversal is effectively impossible.)

To stop deep reorgs outright — and to protect against certain attacks on young chains — systems often add **checkpoints**¹⁶ : specific (height, hash) anchors, baked into the software, declaring "this block at this height is final by decree; no reorg may go behind it." Checkpoints trade a little of the "no authority" purity for a hard floor under history. Hathor uses them (Ch 10, Ch 32).

That is the complete classic picture: a replicated, hash-chained ledger, extended by whoever proves the most work, with conflicts resolved by most-work-wins and finality that hardens with depth. Every piece answers part of the double-spend problem from §6.1. Hathor keeps the goals and changes several of the mechanisms — which is the bridge out of this chapter.

6.8 Bridge — how Hathor answers these questions

Hathor is a cryptocurrency, so it must solve every problem in this chapter. But its central structural choice is different from the classic blockchain: it does not put transactions inside blocks on a single chain. Each forward-pointer tells you where Hathor's answer is detailed.

Bridge — the classic design vs. Hathor (full treatment in the chapters named):

- **Not a chain — a DAG.** Hathor's biggest divergence: ordinary transactions are not bundled into blocks on one line; they are themselves nodes in a **directed acyclic graph**, each linking to earlier transactions. Blocks still exist (they mint coins and pin down ordering), but the ledger is a graph, not a chain. The codebase calls any node of that graph a vertex. This re-answers §6.5 entirely — **Chapter 8**.
- **Two kinds of link.** Where a classic block had one `prev` pointer, a Hathor vertex has two distinct kinds of edge: **parents** (the DAG/confirmation topology, the analogue of §6.5's hash pointers) and **inputs** (which earlier outputs this transaction spends, the double-spend-relevant edge). Keeping these straight is half of understanding the model — **Chapters 8 & 25**.
- **Ownership by UTXO, not balances.** Hathor tracks coins as discrete **unspent transaction outputs**, not as account balances — there is no "balance" field anywhere; your balance is the sum of outputs you can spend. This is how §6.1's "who owns what" is actually recorded — **Chapter 7**.
- **Work measured as weight.** Hathor expresses proof-of-work not as a target to fall under but as a **weight** — a number that is the base-2 logarithm of the work done. "Most work wins" becomes "most **accumulated weight** wins," computed across the DAG. This is §6.6's append-rule, recast — **Chapter 9**.
- **Conflicts resolved by voiding.** When two transactions spend the same output (the double-spend of §6.1, made concrete), Hathor marks the loser **voided** — a first-class metadata flag — and propagates that voiding to everything built on it. This is §6.7's reorg/conflict resolution, generalized to a graph — **Chapters 10 & 32**.
- **Proof-of-authority too.** For private networks, Hathor can drop proof-of-work entirely and use a fixed set of authorized signers (§6.6's third option) — **Chapter 32**.
- **Checkpoints.** Hathor anchors history with (height, hash) checkpoints from its settings profile (the §6.7 mechanism) — recalled from **Chapter 22**, used in **Chapters 10 & 32**.

Recap

Problem (this chapter)	Classic answer	Where Hathor's answer lives
Double-spend (§6.1)	one agreed history	the DAG + voiding — Ch 8, 10
No trusted party (§6.2)	replicate the ledger	the P2P network — Ch 34

Problem (this chapter)	Classic answer	Where Hathor's answer lives
Keep copies honest (§6.5)	hash-chain, tamper-evident	parents/hash pointers — Ch 8, 25
Who appends? (§6.6)	proof-of-work, most-work-wins	weight & accumulated weight — Ch 9
Sybil resistance (§6.6)	cost a scarce resource	PoW / PoA — Ch 9, 32
Which conflict wins? (§6.7)	heaviest branch	voiding by accumulated weight — Ch 32
Is it final? (§6.7)	probabilistic, by depth	+ checkpoints — Ch 10, 32
Who owns what? (§6.1)	a ledger of transfers	UTXO model — Ch 7

A blockchain is the answer to one question — how do strangers agree on who owns what, with no one in charge — assembled from a few parts: replicate the ledger so no one owns it, hash-chain it so no one can secretly rewrite it, and make appending it cost a scarce resource so no one can cheaply out-vote or out-run the honest majority. The price is complexity, latency, and finality that is only ever probabilistic. Hold the problem firmly, because Hathor keeps the problem and rebuilds the solution: the next chapter replaces "a ledger of balances" with the **UTXO model** — the precise way Hathor records who owns what — and Chapter 8 replaces "a chain of blocks" with the **DAG of vertices** that gives this project its shape.

1. The double-spend problem is the risk that a holder of digital money spends the same unit more than once by sending copies of it to different recipients. Solving it — getting everyone to agree which spend is valid — is the core problem of digital currency. ↩
2. A distributed ledger is a transaction record replicated across many independent computers, each holding a full copy, with no central master. Agreement among the copies replaces trust in a single operator. ↩
3. A node is one computer participating in the network, running the protocol software and (for a full node) keeping and verifying a complete copy of the ledger. `hathor-core` is a full-node implementation. ↩
4. A cryptographic hash function maps input of any size to a fixed-size output (the hash), with properties — determinism, one-wayness, avalanche, collision-resistance — that make it usable as a tamper-evident fingerprint. Examples: SHA-256, SHA-3. ↩
5. A digest (or hash) is the fixed-size output of a hash function — here, 256 bits, usually written as 64 hexadecimal characters. It acts as a compact fingerprint of the input. ↩
6. A block is a batch of transactions grouped and added to the ledger together. In classic blockchains, blocks are the units that form the chain; in Hathor, blocks coexist with free-standing transactions in a graph (Ch 8). ↩

7. A hash pointer is a reference to a piece of data that includes that data's hash, so the reference both locates the data and certifies it is unchanged. Chaining blocks by hash pointer is what makes the history tamper-evident. ↩
8. A Sybil attack is the creation of many fake identities by one actor to gain disproportionate influence in a system that counts participants. Open, permissionless networks must make influence cost a scarce resource to resist it. (Named after a case study of a person with many identities.) ↩
9. Proof-of-work is a scheme where adding a block requires finding a value (the nonce) that makes the block's hash meet a hard target — costly to produce, trivial to verify. The work spent is the proof, and it makes rewriting history expensive. ↩
10. A miner is a participant that performs proof-of-work — repeatedly hashing candidate blocks to find a valid nonce — in exchange for a reward (newly minted coins and/or fees) when it succeeds. ↩
11. A nonce ("number used once") is the variable a miner changes on each attempt so that re-hashing the block yields a different hash, searching for one that meets the target. ↩
12. A target is the threshold a block's hash must fall below to be valid. A lower target means fewer valid hashes, so more guessing — i.e. higher difficulty. Hathor expresses the same idea as a weight instead (Ch 9). ↩
13. A fork is a temporary split in the ledger where different nodes hold different competing versions of recent history, usually because two valid blocks appeared at nearly the same time. It resolves when one branch accumulates more work. ↩
14. A reorganization (reorg) is when a node abandons one branch of history for a competing branch that turned out to carry more work, undoing any transactions that existed only in the abandoned branch. ↩
15. Finality is the assurance that a transaction cannot be reversed. In proof-of-work systems it is probabilistic: never absolute, but exponentially more certain as more blocks are built on top (greater depth). ↩
16. A checkpoint is a hard-coded (height, hash) anchor declaring a particular block final by decree, so no reorganization may rewrite history before it. It bounds how deep a reorg can go, trading a little decentralization for safety. ↩